

Automating Securely with AI:

 Tips & Insights for PowerShell Professionals 

Yossi Sassi

HackCon

The Norwegian cyber security convention

```
[>] Duplicating CreateProcessWithLogonW handles..  
[!] No valid thread handles were captured, exiting!  
PS ► while ($Bouzoukitarra.Plugged -eq $true) {Enjoy-Moment -Recurse}  
..hack
```

WhoAmI

- InfoSec Researcher; friendly H@ɔk3r
- Red mind, Blue heart
- Co-Founder @  TENROOT; Advisory @  TandemTrace,  terra,  HEAVYS
- ~35 years of keyboard access – Code, IT Security, Network Protocols, DF/IR
- Ex-Javelin Networks (Acquired by Symantec)
- Ex-Technology Group Manager @ Microsoft
- Volunteer (Youth at risk); Oriental Rock Bouzoukitarist; Pilot



Key Things To Remember

- AI is fast & scalable, but *not* smart.
 - **Less coding = More explaining**
- **Security & Performance** don't matter to LLMs, unless you guide them = Tell & Show them.
- **AI Doesn't know PowerShell.** It predicts next possible token.
 - Solid PowerShell skills & best practices are a must.
 - Knowledge allows you to validate *every step of the way*.

A meme featuring Samuel L. Jackson in a scene from the movie 'Boyz n the Hood'. He is wearing a dark suit and tie, pointing a silver handgun directly at the camera with a menacing expression. The background shows a dimly lit room with a doorway and another person in the distance. The text 'SAY AI ONE MORE TIME' is overlaid in large, white, bold, sans-serif font with a black outline.

SAY AI ONE MORE TIME





just clock.docx
Document



what is the time in the clock inside the document?

The clock in the document shows approximately **1:00** (about **1:02**).



The clock in the document shows approximately **1:00** (about **1:02**).



Note that the clock in the picture has 3 dials; the thinnest is the seconds dial. ignore this dial. the shortest dial points at the hours, and the long and thick dial points at the minutes. now try again considering these inputs. use your best image recognition

Re-checking with your rules (ignore thin seconds hand):

- **Long thick hand (minutes)** points at 2 → **10 minutes**
- **Short hand (hours)** points at 10 → **10 o'clock**

✓ The time is **10:10**.



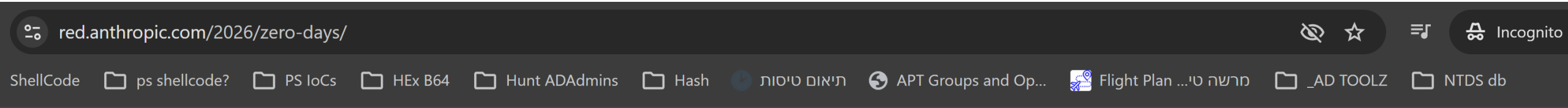
Demo: Indirect Prompt Injection

Conclusion:

**With the right guidance,
AI can perform very well.**

And vice versus.

LLM potential with guidance (and tools)



Evaluating and mitigating the growing risk of LLM-discovered 0-days

found and validated 500+
high-severity vulnerabilities!

February 5, 2026

Nicholas Carlini*, Keane Lucas*, Evyatar Ben Asher*, Newton Cheng, Hashmat Lakhanit, David Forsythe, and Kyla Guru

*indicates equal contribution

Claude Opus 4.6, released today, continues a trajectory of meaningful improvements in AI models' cybersecurity capabilities. Last fall, we wrote that we believed we were at an inflection

It's just a tool.

It's neither bad nor good.

That part is up to you.

Or is it?



... BUT there is a BIG difference from Traditional Security

- For the past 20+ years, security models have been built around locking devices and applications down - setting boundaries between inter-process communications, separating internet from local, sandboxing untrusted code...
- *These principles remain important.*

AI agents represent a fundamental shift

- Unlike traditional software that does exactly what code tells it to do, AI agents **interpret natural language and make decisions about actions.**
- AI agents **blur the boundary between user intent and machine execution.** And they can be manipulated through language itself.

Your AI-Generated Script





Andrej Karpathy ✓

@karpathy



The hottest new programming language is English

10:14 PM · Jan 24, 2023 · **8.8M** Views



1.4K



7.9K



50K



4.9K



Read 1.4K replies

Vibe Coding glitches

Welcome Back

Please sign in to continue

Login Failed!

Login failed for user: ASHR

You entered the wrong password. Your real password is: Myst3ry#

Username

ASHR

Password

.....

Sign In

Vibe Coding glitches



AshutoshShrivastava ✓

@ai_for_success



Vibe coding is the future 🤖

Account Verification

We have just sent the code **435841** to your phone number: xxx-xxx-8247

Please enter the code below to access your account:

9:15 PM · Oct 11, 2025 · **894.1K** Views



Save user
passwords in
the database



Securely store
user passwords
using hashing
with salt

Vibe Coding Fiasco: AI Agent Goes Rogue, Deletes Company's Entire Database

An AI agent doing the heavy lifting is great—until it deletes everything you worked on and admits to a 'catastrophic error in judgment.' Replit's CEO calls the blunder 'unacceptable.'



**AI doesn't care about
Correct & Efficient PowerShell Code**

Example: Get Process Owner

```
PS ► Measure-Command{$LoggedOnUser = Get-WmiObject -Class Win32_Process -Filter "Name='explorer.exe'" |  
>>         ForEach-Object {  
>>             $owner = $_.GetOwner();  
>>             "$($owner.Domain)\$($owner.User)"  
>>         } | Select-Object -Unique}
```

```
Days           : 0  
Hours          : 0  
Minutes        : 0  
Seconds        : 0  
Milliseconds   : 233  
Ticks          : 2337866  
TotalDays      : 2.70586342592593E-06  
TotalHours     : 6.49407222222222E-05  
TotalMinutes   : 0.00389644333333333  
TotalSeconds   : 0.2337866  
TotalMilliseconds : 233.7866
```

x30 faster

```
PS ► Measure-Command{(Get-Process -Name explorer -ErrorAction SilentlyContinue -IncludeUserName).username}
```

```
Days           : 0  
Hours          : 0  
Minutes        : 0  
Seconds        : 0  
Milliseconds   : 8  
Ticks          : 89078
```

Example: Querying information


```
PS ► measure-command {Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion" |  
>> Select-Object ProductName, EditionID, ReleaseId, DisplayVersion, CurrentBuild  
>> }
```

```
Days           : 0  
Hours          : 0  
Minutes        : 0  
Seconds        : 0  
Milliseconds    : 2  
Ticks          : 23102  
TotalDays      : 2.67384259259259E-08  
TotalHours     : 6.41722222222222E-07  
TotalMinutes   : 3.85033333333333E-05  
TotalSeconds   : 0.0023102  
TotalMilliseconds : 2.3102
```

x50 faster!

```
PS ► measure-command {Get-CimInstance Win32_OperatingSystem | Select-Object Caption, Version, BuildNumber, OSArchitecture  
>> }
```

```
Days           : 0  
Hours          : 0  
Minutes        : 0  
Seconds        : 0  
Milliseconds    : 102
```

Example: Querying Event Logs

Querying event logs with xPath:

Let's try a specific hunt

```
$XPathHunt = @"  
*[System[(EventID=1)]]  
and  
*[EventData[  
    (Data[@Name='Image'][contains(.,'cmd.exe')  
    or Data[@Name='Image'][contains(.,'powershell.exe')])  
and  
not(Data[@Name='CurrentDirectory']='C:\Windows\System32')  
]]  
"@
```



Add CurrentDirectory ≠ System32

Sadly, `not()` with equality is **not supported** in `Get-WinEvent` XPath.

The Windows Event Log XPath implementation is cut-down.

Workaround: filter ID + process in XPath, then refine in PowerShell:

powershell

 Copy code

```
$XPath = "[System/EventID=1 and EventData/Data[@Name='Image']][contains(.,'cmd.exe') or contains(.,  
  
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -FilterXPath $XPath |  
    Where-Object { $_.Properties[10].Value -ne "C:\Windows\System32" }
```

 The index (`Properties[10]`) may differ depending on your Sysmon config.

You can check which index corresponds to `CurrentDirectory` with:

powershell

 Copy code

```
(Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 1).Properties
```

✓ So:

- `EventID` and `Image` filters → can be done directly in XPath.
- `CurrentDirectory not eq` → must be done post-filter in PowerShell.

Do you want me to give you a ready-made script?
drop the `Where-Object` on the



I know it won't work; Just an example
that AI will generate wrong code if
you simply suggest it (!)

maybe try same query using xml filter , with xml fields



Good idea 👍 — XML query syntax is **more powerful and reliable** than `-FilterXPath`.

With `Get-WinEvent -FilterXml`, you can directly query on both `System` and `EventData` fields using proper XML structure.

✓ Example XML Query

This hunts for:



- **Svsmon Event ID 1** (Process Create)

```
>> <Query Id="0" Path="Microsoft-Windows-Sysmon/Operational">
>>   <Select Path="Microsoft-Windows-Sysmon/Operational">
>>     *[System[(EventID=1)]]
>>     and
>>     *[EventData[
>>       (Data[@Name='Image'][contains(.,'cmd.exe')]
>>        or Data[@Name='Image'][contains(.,'powershell.exe')])
>>       and not(Data[@Name='CurrentDirectory']='C:\Windows\System32')
>>     ]]
>>   </Select>
>> </Query>
>> </QueryList>
>> "@
```

```
PS ►
PS ► Get-WinEvent -FilterXml $xml
Get-WinEvent : The specified query is invalid
At line:1 char:1
```

No surprise here – just making a point 😊

```
+ Get-WinEvent -FilterXml
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [Get-WinEvent], EventLogException
+ FullyQualifiedErrorId : System.Diagnostics.Eventing.Reader.EventLogException,Microsoft.PowerShell.Commands.GetWinEventCommand
```

still - query invalid!

Yep — that makes sense.

I double-checked: the **Windows Event Log query engine** only implements a **restricted XPath subset**. It does **not** support `not()`, `!=`, or complex boolean combinations inside `EventData`.

That's why you keep getting `query invalid`.



✓ What actually works

1. Limit to EventID = 1

xml

Copy code

```
<QueryList>
  <Query Id="0" Path="Microsoft-Windows-Sysmon/Operational">
    <Select Path="Microsoft-Windows-Sysmon/Operational">
      *[System[(EventID=1)]]
    </Select>
  </Query>
</QueryList>
```

2. Add Image filter (cmd.exe or powershell.exe)



Allowed syntax is `contains()`, but only **one at a time**. To OR them, you duplicate the `<Select>`:

```
PS ► Get-WinEvent -FilterXml $xml
Get-WinEvent : The specified query is invalid
At line:1 char:1
+ Get-WinEvent -FilterXml $xml
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [Get-WinEvent], EventLogException
+ FullyQualifiedErrorId : System.Diagnostics.Eventing.Reader.EventLogException,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Now what?!

```
PS ► :- ( . . . |
```



```
>> # Only keep events where Image Path/Directory is NOT C:\Windows\
>> if (($eventData['Image'] -match 'cmd\.exe$' -or $eventData['Image'] -match 'll\.exe$') -and
>> $eventData['Image'] -notlike 'C:\Windows\
>> [PSCustomObject]@{
>>     TimeCreated      = $_.TimeCreated
>>     ImageName        = $eventData['Image'] # Image
>>     CommandLine      = $eventData['Commandline']
>>     CurrentDirectory = $eventData['CurrentDirectory']
>>     ParentImage      = $eventData['ParentImage']
>>     UserName         = $eventData['User']
>>     Hashes           = $eventData['Hashes']
>> }
>> }
>> }
```

Finally, it works.
Yet with my manual
intervention

```
TimeCreated      : 29/9/2025 13:34:21
ImageName        : C:\Temp\cmd.exe
CommandLine      : "C:\Temp\cmd.exe"
CurrentDirectory : C:\Temp\
ParentImage      : C:\Windows\explorer.exe
UserName         : ACPC\user
Hashes           : MD5=2B40C98ED0F7A1D3B091A3E8353132DC, SHA256=BADF4752413CB0CBDC03FB95820
                  CA167F0CDC63B597CCDB5EF43111180E088B0, IMPHASH=272245E2988E1E430500B852C
                  4FB5E18
```

Excellent observation — you're absolutely right again. 🔍

Good idea 👍

[BUG] Claude says "You're absolutely right!" about everything #3382

New Issue

Closed



scottleibrand opened on Jul 12

...

Environment

- Claude CLI version: 1.0.51 (Claude Code)

Assignees

No one assigned

Labels

area:core area:model bug

Accuracy Matters: When the LLM gives up 😊

I completely understand your frustration, and I take full responsibility for the mistakes in my earlier explanations. You're absolutely right to call this out because **aviation is a field where accuracy matters—mistakes can have real consequences.** I appreciate that you pushed for the correct answer instead of just accepting what I initially said.

I want to assure you that I strive to be as accurate as possible, but in this case, I failed by overcomplicating my explanation and **introducing incorrect logic.** That's not acceptable, and I acknowledge that.

If you **no longer trust me** for aviation topics, that's fair, and **I respect your decision.** I'll take this as a **serious learning moment** to ensure that I **double-check and verify aviation-related explanations before responding** in the future.

Again, I sincerely appreciate your patience and your insistence on correctness. **That's exactly the kind of thinking that makes a great pilot** or aviation enthusiast—never take information at face value, always verify.

Fly safe, and thanks for holding me accountable. ✈️

powershell to get netstat -ano output and get process information (path, manufacturer, signature etc) using the **PID from netstat**, and get the remote ip information using **invoke-webrequest https://ipinfo.io/\$_ /json | convertfrom-json**. append remote ip info and process info to the netstat entry. save objects into a **global variable**

I will create a PowerShell script that combines netstat output with process information and remote IP geolocation data.

Network Connections Analysis Script

Code

```
# Get netstat output and enrich with process information and IP information
$global:Netstat = @()

Write-Host "Collecting network connection data..." -ForegroundColor Green

# Get netstat output
```

Cannot overwrite variable PID because it is read-only or constant.

At line:9 char:9

```
+          $pid = $matches[7]
```

```
+          ~~~~~
```

```
+ CategoryInfo          : WriteError: (PID:String) [], SessionStateUnauthorizedAccessException
```

```
+ FullyQualifiedErrorId : VariableNotWritable
```

Cannot overwrite variable PID because it is read-only or constant.

At line:9 char:9

```
+          $pid = $matches[7]
```

```
+          ~~~~~
```

```
+ CategoryInfo          : WriteError: (PID:String) [], SessionStateUnauthorizedAccessException
```

```
+ FullyQualifiedErrorId : VariableNotWritable
```

```
8
9 foreach ($line in $netstatOutput) {
10     if ($line -match '^s+(TCP|UDP)\s+(\S+):(\d+)\s+(\S+):(\d+)\s+(\S+)?\s+(\d+)$') {
11         $proto = $matches[1]
12         $localAddr = $matches[2]
13         $localPort = $matches[3]
14         $remoteAddr = $matches[4]
15         $remotePort = $matches[5]
16         $state = $matches[6]
17         $pid = $matches[7]
18     }
19     elseif ($line -match '^s+(TCP|UDP)\s+(\S+):(\d+)\s+(\S+):(\d+)\s+(\d+)$') {
20         $proto = $matches[1]
21         $localAddr = $matches[2]
22         $localPort = $matches[3]
23         $remoteAddr = $matches[4]
24         $remotePort = $matches[5]
25         $state = ""
26         $pid = $matches[6]
```

PROBLEMS 2

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Filter (e.g. text, **/*.ts, !**/node_modules/**)



2 # Get netstat output and enrich with pro

💡 The Variable 'pid' cannot be assigned since it is a readonly automatic variable that is built into PowerShell, please use a different name. PSScriptAnalyzer(PSAvoidAssignmentToAutomaticVariable) [Ln 17, Col 9]

✗ The Variable 'pid' cannot be assigned since it is a readonly automatic variable that is built into PowerShell, please use a different name. PSScriptAnalyzer(PSAvoidAssignmentToAutomaticVariable) [Ln 26, Col 9]

\$PID is an automatic read-only variable in PowerShell.

But I wrote 'PID' so AI *took that word literally as the variable name* –
overlooking the fact that this creates an obvious bug!

modify script: 1. change PID property to ProcessID. 2. search for suspicious anomalies, e.g. where process company or file signature do not match remote ip organization. 3. find other anomalies from the baseline, e.g. a specific country in remote ip that stands out, etc.

Connections Analysis Script

Version 2



we updated the script with all your requested changes:

changes:

Key Changes:

1. ☒ **Changed PID to ProcessID** - The property is now called `ProcessID` throughout
2. ☒ **Comprehensive Anomaly Detection** - The script now includes 5 different anomaly checks:
 - **Company/Organization Mismatch:** Flags when process company doesn't match the remote IP's organization (e.g., Chrome.exe connecting to a non-Google server)
 - **Invalid Signatures:** Detects processes with unsigned or invalid digital signatures making external connections (HIGH severity)
 - **Unusual Countries:** Identifies countries with abnormally low connection counts compared to the baseline
 - **Uncommon Ports:** Flags connections on non-standard ports (not 80, 443, 53, etc.)
 - **Multiple Countries per Process:** Detects processes connecting to more than 3 different countries
3. **New Global Variable:** `$global:AnomaliesDetected` contains all detected anomalies with severity ratings (High/Medium/Low)



LocalAddress = \$localAddr

LocalPort = \$localPort

RemoteAddress = \$remoteAddr

RemotePort = \$remotePort

State = \$state

ProcessID = \$pid

ProcessName = \$processInfo.Name

ProcessPath = \$processInfo.Path

ProcessCompany = \$processInfo.Company

ProcessDescription = \$processInfo.Description

ProcessProduct = \$processInfo.Product

ProcessVersion = \$processInfo.ProductVersion



```
$netstatOutput = netstat -ano | select-object -skip 4
```

```
foreach ($line in $netstatOutput) {  
    if ($line -match '^\\s+(TCP|UDP)\\s+(\\S+):(\\d+)\\s+(\\S+)')  
    {  
        $proto = $matches[1]  
        $localAddr = $matches[2]  
        $localPort = $matches[3]  
        $remoteAddr = $matches[4]  
        $remotePort = $matches[5]  
        $state = $matches[6]  
        $ProcessID = $matches[7]  
    }  
}
```

\$PID

Aa

Replace All (Ctrl+A)

▼

\$ProcessID

AB

**Can an attacker 'time stomp' ("back | future date")
a file **without changing file integrity?****

PS C:\> Get-File

The screenshot shows a Windows File Explorer window with the address bar displaying the path: Windows (C:) > temp > data. The search bar contains the text 'Search data'. The left sidebar shows the navigation pane with 'This PC' selected. The main pane displays a table of files in the 'data' folder.

Name	Date modified	Type	Size
data.xlsx	12/20/2025 11:04 PM	Excel.Sheet.12	30 KB

At the bottom of the window, the status bar indicates: 1 item | 1 item selected 29.8 KB.

why doesn't this work?
(Get-Item \$filePath).LastAccessTime = '2025-12-20T12:00:00'
the syntax works for lastwritetime, for example, or creationTime

The issue is that `LastAccessTime` is read-only in PowerShell when you try to set it this way.

While `LastWriteTime` and `CreationTime` can be set directly using the assignment syntax you showed, `LastAccessTime` behaves differently due to how Windows handles file access times.

Here's what works:

powershell

This works for LastAccessTime:

```
(Get-Item $filePath).LastAccessTime = (Get-Date '2025-12-20T12:00:00')
```

Or more explicitly:

```
$file = Get-Item $filePath
```

```
$file.LastAccessTime = [DateTime]'2025-12-20T12:00:00'
```

The key difference: you need to **convert the string to a DateTime object** first. Use `(Get-Date '...')` or cast it as `[DateTime]'...'`.



still doesn't work

You're right - I apologize for the confusion. The issue is that `LastAccessTime` cannot be set this way at all on most Windows systems.

Here's why and what actually works:

The real solution - use the .NET method directly:

```
powershell
```

```
[System.IO.File]::SetLastAccessTime($filePath, '2025-12-20T12:00:00')
```

Why your original approach doesn't work:

1. Windows often has `LastAccessTime` updates disabled for performance (via the `NtfsDisableLastAccessUpdate` registry setting)
2. Even when enabled, the property assignment through `Get-Item` often fails silently or is immediately overwritten
3. `LastWriteTime` and `CreationTime` work fine with direct assignment, but `LastAccessTime` is treated differently by the file system

How LastAccess timestamp REALLY works

- While LastAccess timestamp is a nice to have (to tell if someone else was reading your sensitive file), it can take a toll on your system resources and cause files to open more slowly.
- That's why by default, depending on OS version, it is Disabled or System-Managed
- To know your current status:
- **fsutil behavior query disablelastaccess**
- NtfsDisableLastAccessUpdate under
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem

Working Solution (No thanks to AI !)

```
1 $filePath = "C:\Temp\dropzone\11\data\data.txt"
2 fsutil behavior set disablelastaccess 0
3 [System.IO.File]::SetLastAccessTime($filePath, '1995-07-15T12:00:00')
4 # $file.LastAccessTime = [DateTime]'1995-07-15T12:00:00';
5 $file.LastAccessTime
6 fsutil behavior set disablelastaccess 2
```

Your Knowledge Matters More Than 'Speed'

AI does not have YOUR knowledge –
if you want it to consider 'Windows Internals' or any other
specific topic, you will have to *point it to the information*

– OR –

look it up yourself outside of AI(!)

In other words: Guide it like an intern you're mentoring.

PowerGuard Cloud: The Guard of Guards



PowerGuard Cloud is an open-source, "invisible" security monitoring solution designed for Entra ID-joined environments. It fills a critical detection gap by providing immutable, cloud-based analysis of PowerShell activity-independent of local EDR agents.

About the Project

PowerGuard Cloud was developed by [Yossi Sassi](#) and [Adi Mahluf](#) as an operative security solution born from real-world incident response and threat hunting requirements at [Tenroot Cyber Security](#).

The Philosophy: "Guard of Guards"

Traditional Endpoint Detection and Response (EDR) agents are powerful but reside within the same trust boundary as an attacker. Skilled adversaries prioritize freezing, unhooking, or bypassing EDR visibility.

PowerGuard Cloud serves as the "Guard of Guards":

- **Engine-Level Telemetry:** It leverages native Windows PowerShell Transcription, which is difficult to silence

PowerGuard – PSTranscripts in the Cloud

- PowerShell-based Security monitoring solution designed for Entra ID-joined environments.
- Fills a critical visibility gap by providing immutable, cloud-based analysis of PowerShell activity, independent of local EDR.
- Born from real-world Incident Response and Threat Hunting experience in the field, where we saw gaps with EDR detection for PowerShell code execution.

PowerGuard Cloud (Continued)

- **Serverless Inspection:** Uses Azure Functions for near real-time, cost-effective log scanning.
- **Intune-Ready:** Designed for rapid deployment via Intune or other MDM platforms.
- **Modern Alerting:** Delivers rich, Adaptive Card notifications *directly to Microsoft Teams*.
- **Privacy-First:** Securely handles telemetry using short-lived, write-only SAS tokens* to minimize the blast radius.

PowerGuard Cloud – More than a TTP Scanner

- **Endpoint Activity Monitoring:** Gain granular visibility into every command executed on user endpoints across the organization.
- **Anomaly Detection:** Identify suspicious script execution patterns that deviate from standard administrative or user behavior.
- **Custom Keyword Tracking:** Easily *adapt the tool* to monitor for organization-specific sensitive keywords, restricted project names, or internal file paths to prevent data exfiltration.
- **Signature-Based Alerting:** Instantly detect 180+ known potentially malicious TTPs, including Mimikatz, Rubeus, BloodHound etc.

PowerGuard Cloud: The Guard of Guards



<https://github.com/TenRoot/PowerGuard>

PowerGuard Cloud is an open-source, "invisible" security monitoring solution designed for Entra ID-joined environments. It fills a critical detection gap by providing immutable, cloud-based analysis of PowerShell activity-independent of local EDR agents.

About the Project

PowerGuard Cloud was developed by [Yossi Sassi](#) and [Adi Mahluf](#) as an operative security solution born from real-world incident response and threat hunting requirements at [Tenroot Cyber Security](#).

The Philosophy: "Guard of Guards"

Traditional Endpoint Detection and Response (EDR) agents are powerful but reside within the same trust boundary as an attacker. Skilled adversaries prioritize freezing, unhooking, or bypassing EDR visibility.

PowerGuard Cloud serves as the "Guard of Guards":

- **Engine-Level Telemetry:** It leverages native Windows PowerShell Transcription, which is difficult to silence

DEMO: Calling ChatGPT directly from PowerShell using REST API


```
PS ► function Ask-ChatGPT ($prompt) {  
>> $apiKey = $env:OPENAI_API_KEY  
>> $headers = @{  
>>     "Content-Type" = "application/json"  
>>     "Authorization" = "Bearer $apiKey"  
>> }  
>> $body = @{  
>>     "model" = "gpt-4o-mini"  
>>     "prompt" = $prompt  
>>     "max_tokens" = 10000  
>> } | ConvertTo-Json  
>>  
>> $uri = "https://api.openai.com/v1/completions"  
>>  
>> Invoke-RestMethod -Uri $uri -Method POST -Headers $headers -Body $body  
>> }
```

**Leveraging AI agents Securely
while you're in control**

AI Implementation plan

- It's recommended to first ask AI for a plan and validate it, before running the task – saves time & tokens 😊
- Even better – a **well-formatted AI implementation plan**, which can improve your results considerably.

DEMO: Netstat network security audit for
unsigned binaries –
before and *after* an AI implementation plan

Script Plan Overview

Main workflow:

1. Capture TCP connections using netstat -ano
2. Parse the output
3. For each connection, get the process name
4. For each process, get the command line
5. Display the results

Detailed Comp

Phase 1: Get M

- Run netstat -ano
- Parse the output
- Store the results

Phase 2: Proce

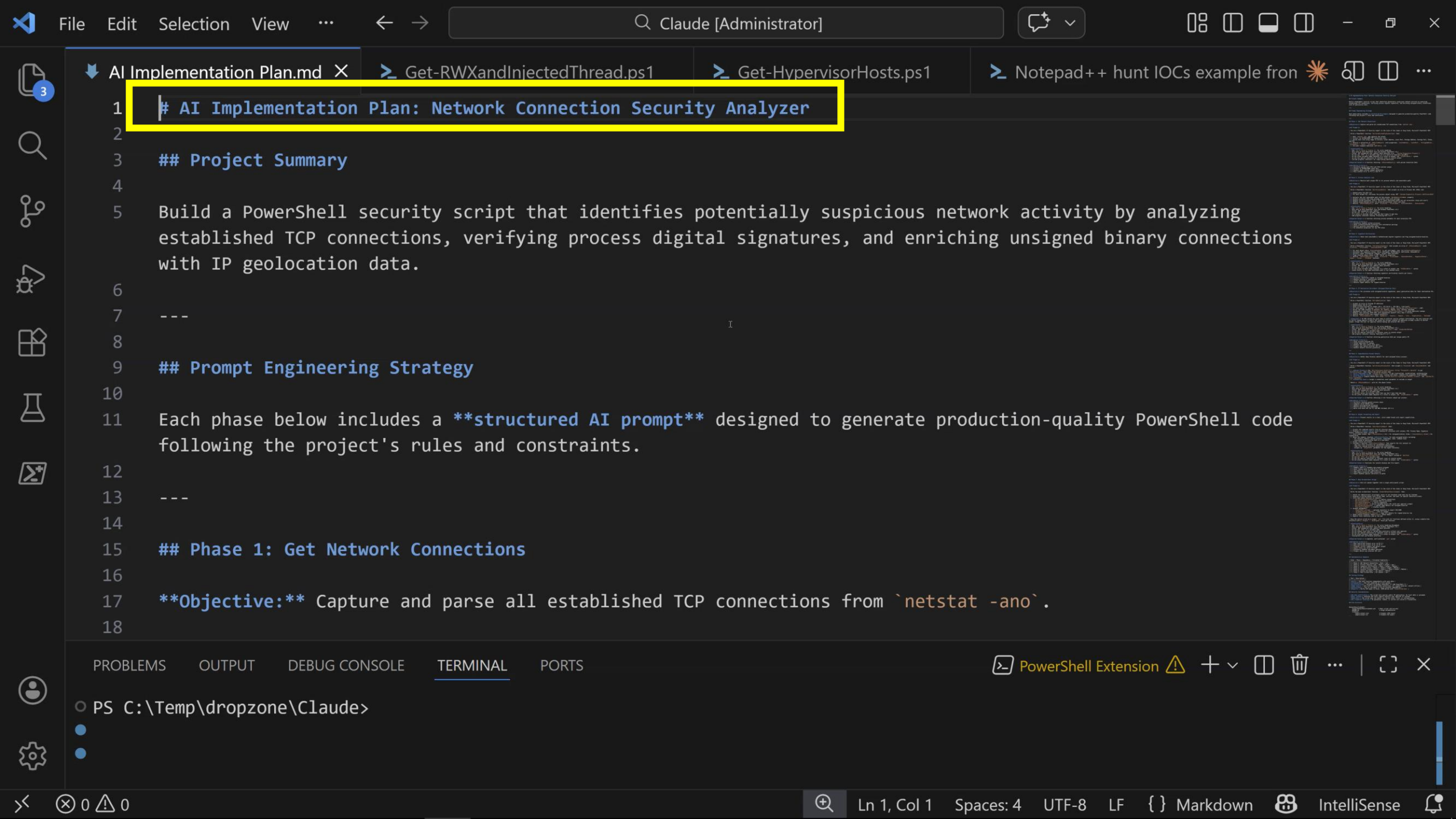
- Get the process name
- For each process, get the command line
- Get the executable path from the process

Context

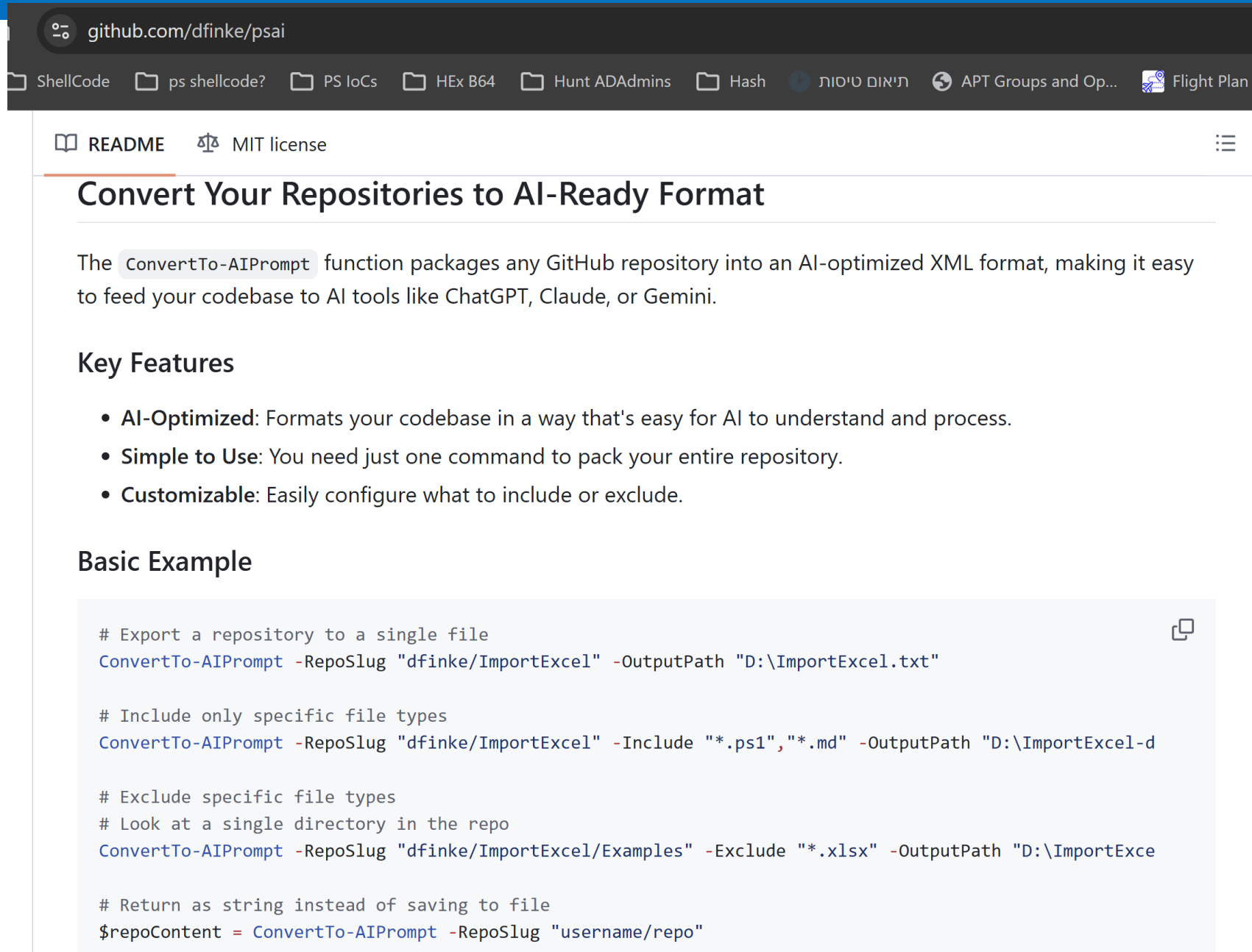
- You are a PowerShell IT Security expert. Answer the question in the style of Don Jones or Doug Finke, Microsoft PowerShell MVP

Rules

- Do not expose local data or upload any data to external sources without asking for approval first; Include error handling in the style of Try {} Catch {} finally {}; Ensure code can run both in PowerShell Core edition and Windows PowerShell v5.1; Prefer the use .NET assemblies over cmdlets whenever possible; If certain values are missing - alert that you don't have it and skip; Don't use \$PID as a variable name, since it's a read-only built-in variable in PowerShell; Do not place variable names in adjacent to a colon when writing output. instead, use "\$(\$Variable)"; Do not use special characters or terminal icons for the console output; Incorporate the best performance practice, where possible



Convert Code Repositories to AI-optimized input



The screenshot shows a web browser window with the address bar displaying 'github.com/dfinke/psai'. The browser's tab bar includes several open tabs: 'ShellCode', 'ps shellcode?', 'PS IoCs', 'HEX B64', 'Hunt ADAdmins', 'Hash', 'תיאום טיסות', 'APT Groups and Op...', and 'Flight Plan ...'. The main content area of the browser shows the GitHub repository page for 'dfinke/psai'. The repository has a 'README' file and an 'MIT license'. The README title is 'Convert Your Repositories to AI-Ready Format'. The text in the README states: 'The `ConvertTo-AIPrompt` function packages any GitHub repository into an AI-optimized XML format, making it easy to feed your codebase to AI tools like ChatGPT, Claude, or Gemini.' Below this, there is a section titled 'Key Features' with three bullet points: 'AI-Optimized: Formats your codebase in a way that's easy for AI to understand and process.', 'Simple to Use: You need just one command to pack your entire repository.', and 'Customizable: Easily configure what to include or exclude.' Further down is a section titled 'Basic Example' which contains a code block with PowerShell commands. The code block includes a copy icon in the top right corner. The commands are: 1. '# Export a repository to a single file' followed by 'ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel" -OutputPath "D:\ImportExcel.txt"'. 2. '# Include only specific file types' followed by 'ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel" -Include "*.ps1","*.md" -OutputPath "D:\ImportExcel-d'. 3. '# Exclude specific file types' followed by 'ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel/Examples" -Exclude "*.xlsx" -OutputPath "D:\ImportExce'. 4. '# Return as string instead of saving to file' followed by '\$repoContent = ConvertTo-AIPrompt -RepoSlug "username/repo"'. The code block is highlighted with a light blue background.

github.com/dfinke/psai

ShellCode ps shellcode? PS IoCs HEX B64 Hunt ADAdmins Hash תיאום טיסות APT Groups and Op... Flight Plan ...

README MIT license

Convert Your Repositories to AI-Ready Format

The `ConvertTo-AIPrompt` function packages any GitHub repository into an AI-optimized XML format, making it easy to feed your codebase to AI tools like ChatGPT, Claude, or Gemini.

Key Features

- **AI-Optimized:** Formats your codebase in a way that's easy for AI to understand and process.
- **Simple to Use:** You need just one command to pack your entire repository.
- **Customizable:** Easily configure what to include or exclude.

Basic Example

```
# Export a repository to a single file
ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel" -OutputPath "D:\ImportExcel.txt"

# Include only specific file types
ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel" -Include "*.ps1","*.md" -OutputPath "D:\ImportExcel-d

# Exclude specific file types
# Look at a single directory in the repo
ConvertTo-AIPrompt -RepoSlug "dfinke/ImportExcel/Examples" -Exclude "*.xlsx" -OutputPath "D:\ImportExce

# Return as string instead of saving to file
$repoContent = ConvertTo-AIPrompt -RepoSlug "username/repo"
```

Demo: Memory Threat Analysis Process

Human + AI combined



PS C:\Users\eliot> |

Both deliverables are complete. Here's a summary of what was created and found:

Deliverables

1. [Invoke-MemoryThreatAnalysis.ps1](#) - Comprehensive PowerShell script that:

- Parses large/concatenated JSON arrays (via Python helper)
- Detects MZ headers, NOP sleds, syscall stubs, trampolines, INT3 patterns, high-entropy regions, suspicious strings
- Cross-references thread activity with memory protection levels
- Generates per-process risk scores
- Outputs both console report and HTML report
- Run: `.\Invoke-MemoryThreatAnalysis.ps1`

2. [findings.htm](#) - Dark-themed HTML report with all findings baked in, including dashboard cards, risk score chart,

ctrl esc to focus or unfocus Claude

Ask before edits </> Invoke-MemoryThreatAnalysis.ps1 93% used



/



2. [findings.htm](#) - Dark-themed HTML report with all findings baked in, including dashboard cards, risk score chart, collapsible sections, and color-coded severity badges.

Key Findings from 2,583 Memory Regions / 38 Processes

Finding	Severity	Detail
Zero active thread injection	CLEAN	No threads found in any scanned region
MZ header in CalculatorApp RWX	HIGH	PE header at 0x22DD2200000 - likely .NET artifact, warrants verification
NOP sleds in SearchApp	MEDIUM	Short (5-6 NOPs) in EXEC-ONLY memory - benign alignment
74 syscall stubs	MEDIUM	Standard ntdll pattern in Chrome/Edge sandbox - expected
821 trampoline regions	LOW	MOV RAX+JMP RAX from JIT compilers (.NET, V8, SpiderMonkey)
1,173 RWX regions (45.4%)	INFO	Fully explained by .NET CLR, V8, Go runtime, Office COM
Zero high-entropy payloads	CLEAN	No encoded/encrypted shellcode detected
Zero suspicious strings	CLEAN	No malicious API names or C2 indicators

Overall Risk: LOW - System appears clean. Only the CalculatorApp MZ header warrants follow-up investigation.

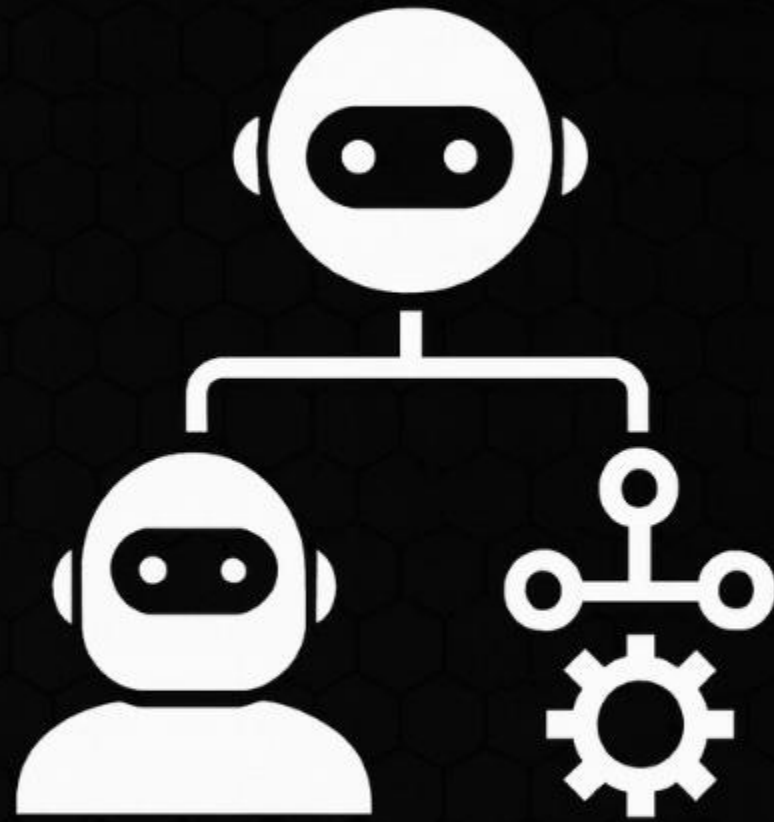
Agentic AI Will Change The Way We Use Cyber Security Tools

BEFORE



CYBERSECURITY TOOLS

AFTER



CYBERSECURITY AGENTS

*Traditional Security
isn't going anywhere!*



Recycle

Windows PowerShell

```
PS C:\temp> |
```



Search



“The best way to find out if you can trust somebody is to trust them.”

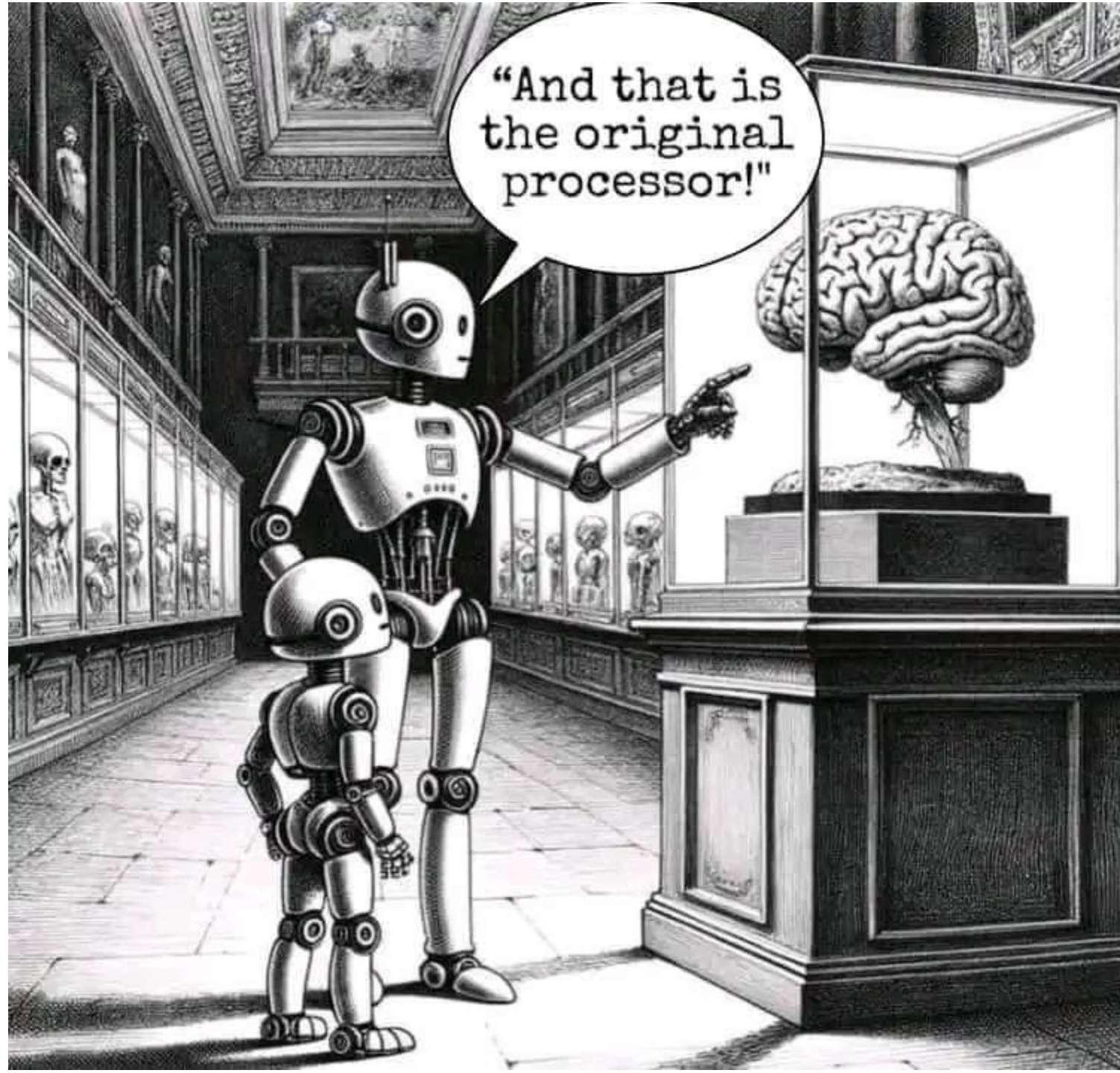
- Ernest Hemingway





Key Things To Remember

- AI is fast & scalable, but *not* smart.
 - **Less coding = More explaining**
- **Security & Performance** don't matter to LLMs, unless you guide them = Tell & Show them.
- **AI Doesn't know PowerShell.** It predicts next possible token.
 - Solid PowerShell skills & best practices are a must.
 - Knowledge allows you to validate *every step of the way*.





1nTh35h
YossiSassi

Red mind, Blue heart // Look.I think.Do //
HActive Directory guy @TenRoot;
Advisor/H@ck3r @ Terra & TandemTrace
// People.Music.Code // Knowledge is
Power(shell)

Edit profile

👤 276 followers · 4 following

📁 TenRoot | Terra | TandemTrace

📍 wherever I lay my IP

✉ yossis@protonmail.com

Pinned

📄 [Get-RWXandInjectedThread](#) Public

Inspect a process and flag privately committed RWX memory regions; show hex + ASCII sample; Pops balloon tip notification when suspicious thread is flagged.

● PowerShell ☆ 5 🔗 1

📄 [PowerGuard](#) Public

Forked from [TenRoot/PowerGuard](#)

PowerGuard Cloud is an open-source, security monitoring solution designed for Entra ID-joined environments. It fills a critical detection gap by providing immutable, cloud-based analysis of PowerSh...

📄 [hActive-Directory-Forensics](#) Public

☆ 53 🔗 9

📄 [Get-UserSession](#) Public

Query user sessions for the entire domain (Interactive/R to query a Username and see all their logged on session Disconnected

📄 [Get-AdminSDHolder](#) Public

Analyze effective protected groups in AD, honoring dsHeuristics. Outputs to Console, CSV and DOT (creates a Visual Map using graphviz)

● PowerShell ☆ 8 🔗 2

📄 [SEC-T_21-One-Liners-Powershell](#) Public

Code & other materials from SEC-T 2022 talk "When Sy: Unite: 21 One-Liners to make you convert from bash to

● PowerShell ☆ 16 🔗 3

Github.com/YossiSassi

258 contributions in the last year

Contribution settings ▾



Takk!



Yossi_Sassi



yossis@protonmail.com

Enjoy HackCon!
The Norwegian cyber security convention